`tree-dp3`: exact cluster-local re-optimization for differential compression of the gapped suffix array

Abstract

The differential compression of a gapped suffix array chooses a set of *links*, each replacing a run of stored positions by a pointer plus a shift. We show that the resulting optimization problem is a maximum-weight set packing subject to exactly two constraints — at most one link per name, and a chosen link's source ranks must all be kept — and that, crucially, *both are pairwise* and there is no acyclicity requirement, because the compressed array stores literal positions and the decoder never recurses. Pairwise feasibility licenses an exact *local* optimization: freeze the links of all but a bounded set of names and re-solve that set optimally. We prove the composition theorem that makes this sound — a link's covered ranks always lie in its own interval, so a frozen link can never drop a rank belonging to a cluster member, and the entire interface to the frozen part collapses to two conditions — and identify the correct notion of dependency between names (one's link sources from the other's interval), showing why the two tempting alternatives (competition for covered ranks, sharing a source) are not conflicts. The resulting algorithm, `tree-dp3`, runs the existing preference-forest DP and unpin/retarget loop and then performs branch-and-bound over bounded dependency clusters. It repairs both known counterexamples to the forest DP, attains the ILP optimum on 9 of 10 fixed test instances, and on a 139-configuration suite at $a_{\max} = 16$ reduces total $|C|$ from 252329 to 213078 (15.5 %), improving 122 configurations and worsening none. At the default $a_{\max} = 8$ the same exact solve buys only 3 positions suite-wide: the binding constraint is the candidate universe, not the neighbourhood size.

Contents

1 The compression problem

1.1 Setting and notation

We reuse the setting of the walkthrough (`dep_order_walkthrough`) and of the pseudoforest note (`exact_pseudoforest_dp`) without change. Fix a binary shape S of span s over a text $T[0..n)$. The DisLex transform lays the gapped k-mer names out grouped by residue class $p \bmod s$; the suffix array of the resulting lextext is the **gapped suffix array** (GSA), with $m = n + 1$ ranks. We write $\mathrm{orig}(r)$ for the original text position of rank r, and $\mathrm{name}(r)$ for the gapped k-mer at $\mathrm{orig}(r)$, which is the first symbol of the rank-r gapped suffix.

Because the GSA is sorted by gapped suffix, the ranks carrying a fixed name c form a contiguous **interval**

$$I_c = [\ell_c, r_c) \subseteq \{0, \dots, m-1\}, \qquad \mathrm{isize}(c) = |I_c| = r_c - \ell_c.$$

The intervals $\{I_c\}_c$ *partition* $\{0, \dots, m-1\}$: every rank has exactly one first symbol. We write $\mathcal{N}$ for the set of names c with $\mathrm{isize}(c) > 1$; singletons are never compressed. Throughout, $a_{\max}$ is the maximum shift considered (`-max-add`, default 8) and $\kappa = 3$ is the minimum coverage (`kMinCoverage`).

1.2 What is stored, and what a link is

Compression drops a subset of ranks. The surviving positions, in rank order, form the **compressed array** C. For each name c the hash table stores at most two entries,

$$H_{\mathrm{offset}} = (pos, a, num), \qquad H_{\mathrm{rest}} = (pos', num'),$$

decoding to

$$\mathrm{positions}(c) \;=\; \big\{\, C[pos + t] + a \cdot s \;:\; 0 \le t < num \,\big\} \;\cup\; \big\{\, C[pos' + t] \;:\; 0 \le t < num' \,\big\}. \quad (1)$$

Everything the compressor decides is which H_{offset} entries to install. We call one such decision a *link*.

Definition 1 (Link)**.** A **link** is a quadruple $k = (c_k, a_k, S_k, T_k)$ where c_k is a name, $1 \le a_k \le a_{\max}$ is the *add*, $S_k = [\sigma_k, \tau_k)$ is a contiguous range of ranks (the **source run**), and $T_k = (t_0, \dots, t_{|S_k|-1})$ is the sequence of **covered ranks**, subject to

(V1) for every $0 \le j < |S_k|$: $\mathrm{orig}(t_j) = \mathrm{orig}(\sigma_k + j) + a_k \cdot s$ and $\mathrm{name}(t_j) = c_k$ — so $T_k \subseteq I_{c_k}$, and the map $\sigma_k + j \mapsto t_j$ is a bijection $S_k \to T_k$ that is order-preserving;

(V2) $S_k \cap T_k = \emptyset$;

(V3) $\mathrm{cov}(k) := |T_k| = |S_k| \geq \kappa$.

We write $\mathcal{K}$ for the set of all links and $\mathcal{K}_c = \{k \in \mathcal{K} : c_k = c\}$.

Condition (V1) is the note's order-preservation argument: prepending a symbols to a gapped suffix preserves relative order and shifts the original position by exactly $+a \cdot s$. It is checked in the code by `pred_lexpos_` / `succ_lexpos_`. Condition (V2) is an intrinsic validity condition, not an interaction between two links: a link may not use as a source a rank it itself drops. It is what `emit_intra_runs_` enforces by a two-pointer scan, and what the `clash` test of `enumerate_all_links_` enforces in the source-centric enumeration. Note that (V2) does *not* require $S_k \cap I_{c_k} = \emptyset$: intra-interval links, whose source run overlaps the target's own interval but avoids the ranks it covers, are perfectly legal, and `GCSA_INTRA_LINKS=0` exists only to restore the historical, stricter rule.

1.3 The objective and the two constraints

Let $X \subseteq \mathcal{K}$ be a set of accepted links. A rank is dropped from C exactly when it is covered by some $k \in X$. Hence

$$|C|(X) \;=\; m \;-\; \mathrm{cov}(X), \qquad \mathrm{cov}(X) := \sum_{k \in X} \mathrm{cov}(k), \tag{2}$$

provided the covered sets are disjoint — which Proposition 1 below shows is automatic. Minimizing $|C|$ is therefore maximizing total coverage.

Definition 2 (Feasibility)**.** $X \subseteq \mathcal{K}$ is **feasible** iff

(F1) **one link per name**: $c_k \neq c_{k'}$ for all distinct $k, k' \in X$;

(F2) **source integrity**: $S_k \cap T_{k'} = \emptyset$ for all $k, k' \in X$ with $k \neq k'$.

We write $\mathcal{F}$ for the family of feasible sets.

(F1) is a storage restriction: a `HashEntry` holds a single H_{offset}. (F2) says a source must be physically present in C: if some other accepted link dropped a rank of S_k, then $C[pos + t]$ in (1) would silently read a different rank's position. In the implementation, (F2) is what `run_kept_` verifies before an accept, and what the `pin_count_` array tracks incrementally.

Proposition 1 (Target-disjointness is free)**.** *If X satisfies (F1) then the covered sets $\{T_k\}_{k \in X}$ are pairwise disjoint, and* (2) *holds exactly.*

Proof. Let $k \neq k' \in X$. By (F1), $c_k \neq c_{k'}$. By (V1), $T_k \subseteq I_{c_k}$ and $T_{k'} \subseteq I_{c_{k'}}$, and distinct names have disjoint intervals. Hence $T_k \cap T_{k'} = \emptyset$, so $|\bigcup_{k \in X} T_k| = \sum_{k \in X} \mathrm{cov}(k)$. □

This small observation is the seed of everything that follows: because a link's covered ranks are confined to its own interval, and (F1) makes the per-name decision unique, *the name owns its ranks.* We will use this twice more, in Lemma 5 and Theorem 8.

1.4 There is no acyclicity constraint

It is tempting to expect a third constraint of the form "the points-to relation among names must be acyclic", by analogy with delta-encoded or recursively-defined structures. There is none.

Proposition 2 (Non-recursive decoding)**.** *For any feasible X, the decoder* (1) *is well defined and terminates in $O(num + num')$ time for every name, independently of the structure of X.*

Proof. C stores *literal* text positions of kept ranks in rank order. Given $k \in X$, (F2) guarantees every rank of S_k is kept, so S_k occupies $|S_k|$ consecutive entries of C starting at some index *pos*; this index is exactly what is written into H_{offset}. Evaluating (1) therefore reads *num* consecutive entries of C and adds $a \cdot s$ to each. No term on the right-hand side of (1) refers to positions(c') for any name c'; the definition is not recursive, so there is nothing to be well-founded. □

A hypothetical scheme in which H_{offset} could point at *dropped* (virtual) ranks would decode a chain $c_1 \to c_2 \to \cdots$, and would need that chain to terminate — an acyclicity condition, and a global, non-pairwise one. Constraint (F2) forbids such chains outright, and in exchange the feasible region stays simple:

Lemma 3 (Pairwise feasibility)**.** *$X \in \mathcal{F}$ if and only if every $k \in X$ satisfies (V1)–(V3) and every pair $\{k, k'\} \subseteq X$ with $k \neq k'$ satisfies (F1) and (F2). Consequently $\mathcal{F}$ is downward closed: $X \in \mathcal{F}$ and $Y \subseteq X$ imply $Y \in \mathcal{F}$.*

Proof. Both (F1) and (F2) are universally quantified over ordered pairs of distinct elements of X and involve no other member of X; so the conjunction over X is exactly the conjunction over its 2-element subsets. Downward closure is immediate, since a conjunction over the pairs of Y is a sub-conjunction of the one over the pairs of X. □

Corollary 4 (Set-packing formulation)**.** *Differential compression is the maximum-weight independent set problem*

$$\min_{X \in \mathcal{F}} |C|(X) \;=\; m - \max_{X \text{ independent in } G} \sum_{k \in X} \operatorname{cov}(k),$$

on the conflict graph $G = (\mathcal{K}, E)$ with $\{k, k'\} \in E$ iff $c_k = c_{k'}$, or $S_k \cap T_{k'} \neq \emptyset$, or $S_{k'} \cap T_k \neq \emptyset$.

Corollary 4 is the same formulation the pseudoforest note starts from; what is new here is the emphasis on Lemma 3. Pairwise-ness is precisely the licence for *cluster-local exact optimization*: if feasibility were a global property of X (an acyclicity or an ordering condition), freezing part of a solution and re-solving the rest could produce a globally infeasible result even when every local check passed. Because it is pairwise, the interaction between a frozen part and a re-solved part is fully described by a finite set of two-element checks, and those checks can be precomputed once from the frozen part — which is exactly what §4 does.

2 Background: the forest DP and why it is not optimal

2.1 Recap of `tree-dp`

`tree-dp` (documented in `dep_order_walkthrough`, §7) proceeds as follows.

1. **Preferences.** For each name c with isize(c) > 2, enumerate $\hat{\mathcal{K}}_c$ — the heuristics' view of the link universe (`enumerate_candidates_`: maximal source runs with internal lcp $\geq a+1$) — and record the **preferred** link $\pi(c)$, of maximum coverage with ties broken towards larger a. Preferences ignore availability; they are a static, structural notion.

2. **Preference graph.** Draw a directed edge $c \to w$ whenever $\pi(c)$'s source lies inside I_w. Since $S_{\pi(c)}$ has lcp $\geq a+1 \geq 2$ throughout, it lies inside a single interval, so this graph has out-degree ≤ 1: as an undirected graph it is a pseudoforest.

3. **Cycle-breaking.** The implementation processes preferences in ascending name order and installs parent(c) := w only if c is not already an ancestor of w (`has_ancestor`). Edges that would close a cycle are *discarded*. The result is a forest.

4. **KEEP/COMPRESS DP.** With *src_ok* recording whether the node's preferred source is still available,

$$\mathrm{DP}(v,\mathsf{true}) = \min\Big(\mathrm{isize}(v) + \sum_{w\in\mathrm{ch}(v)} \mathrm{DP}(w,\mathsf{true}),\ \ \mathrm{isize}(v) - \mathrm{cov}(\pi(v)) + \sum_{w\in\mathrm{ch}(v)} \mathrm{DP}(w,\mathsf{false})\Big),$$

$$\mathrm{DP}(v,\mathsf{false}) = \mathrm{isize}(v) + \sum_{w\in\mathrm{ch}(v)} \mathrm{DP}(w,\mathsf{true}),$$

with KEEP on ties. A root may COMPRESS only when its preferred source lies outside its own tree.

5. **Materialize, leftover, Phase II.** The DP's choices are accepted deepest-first; names that never entered the forest are handled by a size-ordered availability greedy; then the shared Phase II unpin/retarget fixed point runs.

The DP is exact *for the model it is given*. Two restrictions of that model cost optimality: each name is allowed only its single preferred link, and each name is allowed only its single forest parent, with any surplus preference edge silently deleted in step 3. The two counterexamples below isolate the two failures.

2.2 Counterexample I: cycle-breaking fixes the wrong orientation

Take $T = (\texttt{AC})^6 = \texttt{ACACACACACAC}$, $n = 12$, $S = \#.\#$, $s = 3$, $m = 13$. Position p carries the name `AA` for even $p \le 8$ and `CC` for odd $p \le 9$; the three tail positions give the singletons `A$`, `C$`, `$$`. The GSA is

rank r	0	1	2	3	4	5	6	7	8	9	10	11	12
orig(r)	12	10	8	6	4	2	0	11	9	7	5	3	1
name(r)	$$	A$	AA	AA	AA	AA	AA	C$	CC	CC	CC	CC	CC

so $I_{\texttt{AA}} = [2, 7)$ and $I_{\texttt{CC}} = [8, 13)$, both of size 5, and $m = 13 = 5 + 5 + 3$.

The whole link universe. A link shifts original positions by $+a \cdot s = 3a$. Since AA sits on even and CC on odd positions, $3a$ must be odd, i.e. a odd, and $a \ge 3$ overshoots the text. So $a = 1$ throughout, and the universe (as reported by `ilp_baseline`: `candidates=4`) is

link	target	a	source S_k	cov	covered T_k
k_1	AA	1	$[10, 13)$, i.e. orig $= 5, 3, 1$	3	$(2, 3, 4)$, i.e. orig $= 8, 6, 4$
k_2	CC	1	$[3, 7)$, i.e. orig $= 6, 4, 2, 0$	4	$(8, 9, 10, 11)$, i.e. orig $= 9, 7, 5, 3$
k_2'	CC	1	$[3, 6)$	3	$(8, 9, 10)$
k_2''	CC	1	$[4, 7)$	3	$(9, 10, 11)$

(k_2', k_2'' are the two sub-runs of k_2 of length $\ge \kappa$; only k_1 and k_2 are maximal, hence only those two are visible to `enumerate_candidates_`.)

Every feasible set has coverage at most 4. $S_{k_1} = \{10, 11, 12\}$ meets $T_{k_2} = \{8, 9, 10, 11\}$ in $\{10, 11\}$, meets $T_{k_2'}$ in $\{10\}$ and $T_{k_2''}$ in $\{10, 11\}$; so k_1 conflicts with every CC link by (F2). Symmetrically $S_{k_2} = \{3, 4, 5, 6\}$ meets $T_{k_1} = \{2, 3, 4\}$ in $\{3, 4\}$. At most one of the two names can compress, and the best single choice is k_2:

$$|C|^\star = 13 - 4 = 9,$$

which `ilp_baseline` confirms (`optimal |C| = 9`, 1 selected candidate).

What `tree-dp` does. The preferred links are $\pi(\texttt{AA}) = k_1$ (cov 3) and $\pi(\texttt{CC}) = k_2$ (cov 4), giving a 2-cycle $\texttt{AA} \leftrightarrow \texttt{CC}$ in the preference graph. Both names have isize $= 5 > 2$ and both preferences clear cov $\geq \kappa$, so, unlike on the $\texttt{GCCTTTAAAG}^3$ instance of the walkthrough, the cycle is *not* filtered away and the cycle-breaking rule fires. Names are processed in ascending order, and $\texttt{AA} < \texttt{CC}$ as arithmetic codes ($2 < 8$), so:

step	edge	outcome
1	$\texttt{AA} \to \texttt{CC}$	installed: parent(AA) = CC
2	$\texttt{CC} \to \texttt{AA}$	**discarded**: `has_ancestor(AA, CC)` is true, so this edge would close a cycle

The forest is the single edge $\texttt{CC} \to \texttt{AA}$ with root CC. The DP then evaluates

$$\begin{aligned} \mathrm{DP}(\texttt{AA}, \mathsf{true}) &= \min(5,\ 5-3) = 2 \quad \text{(COMPRESS)}, \\ \mathrm{cost}_{\mathrm{keep}}(\texttt{CC}) &= 5 + 2 = 7, \end{aligned}$$

and COMPRESS is *not available* at the root: CC's preferred source AA is a forest descendant, which is exactly the special case the root rule patches. So CC is forced to KEEP and the DP returns 7, i.e.

$$|C| \ = \ 7 + 3 \text{ singletons } = \ 10.$$

The subsequent leftover greedy and Phase II find nothing (both names are saturated), so `tree-dp` = `tree-dp2` = 10 against the optimum 9.

Diagnosis. The instance contains a genuine, high-coverage mutual preference, and the two orientations are *not* equally good: $\texttt{CC} \leftarrow \texttt{AA}$ is worth 4, $\texttt{AA} \leftarrow \texttt{CC}$ only 3. Cycle-breaking does not compare them; it keeps whichever edge it meets first in a deterministic but semantically arbitrary order, and the DP then optimizes exactly — inside a model from which the better option has already been removed. The pseudoforest DP of `exact_pseudoforest_dp` fixes precisely this failure by folding the cycle instead of cutting it. Note that plain `dep-order` happens to get 9 here (its Phase II retarget lets CC take the coverage-4 link), while size-order greedy gets 10: no method below is uniformly better than another, which is itself part of the motivation for an exact local solve.

2.3 Counterexample II: the optimum is not built from preferred links

Take $T = (\texttt{ACGT})^5$, $n = 20$, $S = \#.\#$, $m = 21$. The four compressible names and their preferred links are

name c	isize	I_c	$\pi(c)$ sources from	cov	detail
AG	5	$[1, 6)$	CT	4	$a = 1$, $S = [7, 11)$, $T = (1, 2, 3, 4)$
CT	5	$[6, 11)$	GA	4	$a = 1$, $S = [12, 16)$, $T = (6, 7, 8, 9)$
GA	4	$[12, 16)$	AG	3	$a = 2$, $S = [3, 6)$, $T = (12, 13, 14)$
TC	4	$[17, 21)$	AG	4	$a = 1$, $S = [2, 6)$, $T = (17, 18, 19, 20)$

plus three singletons (\$\$, G\$, T\$), so $m = 5 + 5 + 4 + 4 + 3 = 21$.

The preference graph has a 3-cycle $\texttt{AG} \to \texttt{GA} \to \texttt{CT} \to \texttt{AG}$ (reading "$w \to c$" as "c prefers a source in I_w") with TC hanging off AG. Cycle-breaking in ascending name order installs parent(AG) = CT, then parent(CT) = GA, then *discards* $\texttt{GA} \to \texttt{AG}$ (which would close the cycle), then installs parent(TC) = AG. The forest is the path

$$\texttt{GA} \ \to \ \texttt{CT} \ \to \ \texttt{AG} \ \to \ \texttt{TC},$$

rooted at GA, exactly as GCSA_TRACE_DP=1 reports. The DP unrolls to

$$\begin{aligned} \mathrm{DP}(\mathtt{TC},\mathsf{true}) &= \min(4,\ 4-4) = 0, & \mathrm{DP}(\mathtt{TC},\mathsf{false}) &= 4,\\ \mathrm{DP}(\mathtt{AG},\mathsf{true}) &= \min(5+0,\ 1+4) = 5, & \mathrm{DP}(\mathtt{AG},\mathsf{false}) &= 5+0 = 5,\\ \mathrm{DP}(\mathtt{CT},\mathsf{true}) &= \min(5+5,\ 1+5) = 6, & \mathrm{cost}(\mathtt{GA}) &= 4+6 = 10, \end{aligned}$$

where AG takes KEEP on the tie, CT takes COMPRESS, and the root GA has no COMPRESS option at all because its preferred source AG is a forest descendant. The DP therefore accepts two links — TC←AG (cov 4) and CT←GA (cov 4) — for total coverage 8 and

$$|C| \;=\; 21-8 \;=\; 13.$$

The ILP optimum is 11, i.e. coverage 10, realised by *three* links, all sourcing from the (fully kept) interval $I_{\mathtt{AG}}$:

target	source	a	cov	source ranks	preferred?
TC	AG	1	4	$[2,6)$	yes
GA	AG	2	3	$[3,6)$	yes — but this is the edge cycle-breaking deleted
CT	AG	3	3	$[3,6)$	**no** ($\pi(\mathtt{CT})$ is CT←GA at $a=1$, cov 4)

Two things go wrong for the forest model simultaneously. First, one of the three links is on the cycle edge that step 3 discarded. Second, and more fundamentally, the optimal solution asks CT to take a *worse* link than its preferred one (cov 3 at $a=3$ instead of cov 4 at $a=1$), because doing so moves its source off $I_{\mathtt{GA}}$ and onto $I_{\mathtt{AG}}$, freeing GA to compress in turn. No amount of cycle-folding recovers this: the model that allows one link per name is not rich enough, and a genuinely non-preferred add must be considered.

Note also that the three optimal links all read from the same ranks $[3,6) \subseteq S_{\mathtt{TC}} = [2,6)$. That is not an accident, and it is the phenomenon Lemma 5 (b) below is about: sources are shared for free.

3 The dependency relation

We now identify exactly when two names can constrain one another. This is the relation whose connected balls will become the clusters.

Definition 3 (Dependency)**.** Two distinct names $c, d \in \mathcal{N}$ are **dependent**, written $c \sim d$, iff some link of one draws its source from the other's interval:

$$c \sim d \quad :\Longleftrightarrow \quad \big(\exists\, k \in \hat{\mathcal{K}}_c :\ S_k \cap I_d \neq \emptyset\big) \ \text{ or } \ \big(\exists\, k \in \hat{\mathcal{K}}_d :\ S_k \cap I_c \neq \emptyset\big).$$

The **dependency graph** is $D = (\mathcal{N}, \sim)$.

In the implementation this graph is built by walking, for every cached candidate of every name c, the ranks of its source run and adding the undirected edge $\{c, \mathrm{name}(r)\}$ (deduplicated, and with consecutive equal names skipped, since a source run typically lies in one interval).

Lemma 5 ($\sim$ is the only binding relation)**.** *Let $c \neq d$ be names with $c \not\sim d$. Then for every $k \in \hat{\mathcal{K}}_c$ and every $k' \in \hat{\mathcal{K}}_d$, the pair $\{k, k'\}$ is feasible. Consequently, in the conflict graph of Corollary 4, there is no edge between $\hat{\mathcal{K}}_c$ and $\hat{\mathcal{K}}_d$.*

Proof. (F1) cannot bind, since $c_k = c \neq d = c_{k'}$. For (F2), consider $S_k \cap T_{k'}$. By (V1), $T_{k'} \subseteq I_d$, and by $c \not\sim d$ we have $S_k \cap I_d = \emptyset$; hence $S_k \cap T_{k'} = \emptyset$. The symmetric argument, using $T_k \subseteq I_c$ and $S_{k'} \cap I_c = \emptyset$, gives $S_{k'} \cap T_k = \emptyset$. □

Two other relations look like conflicts and are not. Getting either one wrong would silently enlarge the clusters (making the algorithm slower without making it better) or, worse, suggest constraints that do not exist.

Lemma 6 (Two non-conflicts)**.** *Let k, k' be links with $c_k \neq c_{k'}$.*

(a) Names never compete for covered ranks: $T_k \cap T_{k'} = \emptyset$ *always.*

(b) Sharing a source is not a conflict: $S_k \cap S_{k'}$ *may be arbitrary — in particular $S_k = S_{k'}$ is allowed — without affecting feasibility of $\{k, k'\}$.*

Proof. (a) is Proposition 1: $T_k \subseteq I_{c_k}$, $T_{k'} \subseteq I_{c_{k'}}$, and distinct names have disjoint intervals. Note this holds even for intra-interval links, whose *sources* may straddle interval boundaries; only the covered set is confined.

(b) Definition 2 constrains S_k against $T_{k'}$ and $S_{k'}$ against T_k; the pair $(S_k, S_{k'})$ does not occur in either (F1) or (F2). Semantically: an accepted link only ever *reads* the entries of C indexed by its source run, and reading is not exclusive. This is why `pin_count_` is a counter rather than a lock — a rank may be pinned by many links at once, and `revoke_` merely decrements. □

Remark 1. Counterexample II (§2.3) is a witness for Lemma 6 (b) with three links, not two: the optimal solution has `TC`, `GA` and `CT` all sourcing from $I_{\texttt{AG}}$, with $S_{\texttt{GA}} = S_{\texttt{CT}} = [3, 6)$ literally equal and $S_{\texttt{TC}} = [2, 6)$ a superset. Had we modelled source-sharing as a conflict, that solution would have been declared infeasible and the algorithm would have reproduced `tree-dp`'s 13.

Remark 2 ($\sim$ is necessary but not sufficient). Definition 3 quantifies over the whole candidate list of each name, not over the chosen link, so $c \sim d$ does not imply that the *selected* links conflict. This is deliberate and conservative: the cluster must contain every name whose choice could possibly interact with a member's choice, so $\sim$ over-approximates. Real conflicts are then tested exactly, per pair of concrete links, during the enumeration (§5).

4 The composition theorem

We can now state the result that makes exact local re-optimization sound. Fix a feasible X and a set of names $P \subseteq \mathcal{N}$ (the **cluster**). Write

$$X_P = \{k \in X : c_k \in P\}, \qquad F = X \setminus X_P \quad \text{(the \textbf{frozen} links)},$$

and define the two *interface sets* of the frozen part,

$$\mathrm{Drop}(F) = \bigcup_{f \in F} T_f, \qquad \mathrm{Src}(F) = \bigcup_{f \in F} S_f.$$

Lemma 7 (Cluster ownership)**.** *Let $R_P = \bigcup_{c \in P} I_c$ be the ranks owned by the cluster. Then $\mathrm{Drop}(F) \cap R_P = \emptyset$. That is, no frozen link can drop a rank of a cluster member's interval; whether a rank of R_P is kept or dropped is decided entirely by the links chosen for P.*

Proof. Let $f \in F$. By definition of F, $c_f \notin P$. By (V1), $T_f \subseteq I_{c_f}$. Since the intervals partition the ranks and $c_f \neq c$ for every $c \in P$, we get $T_f \cap I_c = \emptyset$ for every $c \in P$, hence $T_f \cap R_P = \emptyset$. □

Theorem 8 (Composition)**.** *Let $X \in \mathcal{F}$, $P \subseteq \mathcal{N}$, $F = X \setminus X_P$ as above. Let Y be any set of links such that*

(C1) every $y \in Y$ satisfies (V1)–(V3) and $c_y \in P$, and Y contains at most one link per name;

(C2) every pair of distinct $y, y' \in Y$ satisfies (F2);

(C3) $S_y \cap \mathrm{Drop}(F) = \emptyset$ for every $y \in Y$ ("do not source from a rank a frozen link drops");

(C4) $T_y \cap \mathrm{Src}(F) = \emptyset$ *for every* $y \in Y$ *("do not drop a rank a frozen link uses as a source").*

Then $F \cup Y \in \mathcal{F}$ *and*

$$|C|(F \cup Y) \;=\; |C|(X) \;+\; \mathrm{cov}(X_P) \;-\; \mathrm{cov}(Y).$$

Conversely, every feasible $Z \in \mathcal{F}$ *with* $Z \setminus Z_P = F$ *satisfies (C1)–(C4) with* $Y = Z_P$*. Hence (C1)–(C4) characterise exactly the completions of* F*, and*

$$\min\big\{\, |C|(Z) \;:\; Z \in \mathcal{F},\; Z \backslash Z_P = F \,\big\} \;=\; |C|(X) + \mathrm{cov}(X_P) - \max\big\{\, \mathrm{cov}(Y) : Y \text{ satisfies (C1)–(C4)} \,\big\}.$$

Proof. Feasibility. We check (F1) and (F2) for $F \cup Y$ by Lemma 3, i.e. pair by pair. Pairs inside F are feasible because $F \subseteq X \in \mathcal{F}$ and $\mathcal{F}$ is downward closed. Pairs inside Y are (C1) and (C2). For a mixed pair $f \in F$, $y \in Y$: (F1) holds because $c_f \notin P \ni c_y$; (F2) requires $S_y \cap T_f = \emptyset$, which is (C3), and $S_f \cap T_y = \emptyset$, which is (C4).

Objective. By Proposition 1 applied to the feasible sets X and $F \cup Y$, the coverages add, so $|C|(F \cup Y) = m - \mathrm{cov}(F) - \mathrm{cov}(Y)$ and $|C|(X) = m - \mathrm{cov}(F) - \mathrm{cov}(X_P)$; subtracting gives the claim.

Converse. Let $Z \in \mathcal{F}$ with $Z \setminus Z_P = F$ and set $Y = Z_P$. (C1) is (V1)–(V3) plus (F1) restricted to Z_P; (C2) is (F2) restricted to Z_P; and (C3), (C4) are (F2) applied to the mixed pairs of Z, since $\mathrm{Drop}(F)$ and $\mathrm{Src}(F)$ are unions over F and (F2) is quantified pairwise. □

Theorem 8 says the interface between a cluster and the rest of the world is exactly two conditions — and note what is *absent* from that list. There is no condition of the form "T_y must not collide with T_f": by Lemma 7 it is vacuous, because a frozen link's covered ranks live in a non-cluster interval. Nor is there any condition on S_y versus S_f, by Lemma 6 (b). And there is no global condition at all — no ordering, no acyclicity — by Proposition 2. That is what reduces an exact re-solve to a finite enumeration whose only external input is the pair of static bitmasks $\mathrm{Drop}(F)$, $\mathrm{Src}(F)$.

Corollary 9 (Never worse)**.** $Y = X_P$ *satisfies (C1)–(C4). Hence if* $Y^\star$ *maximises* $\mathrm{cov}(Y)$ *subject to (C1)–(C4), then* $\mathrm{cov}(Y^\star) \geq \mathrm{cov}(X_P)$ *and*

$$|C|\big(F \cup Y^\star\big) \;\leq\; |C|(X).$$

An accepted cluster solve can never increase $|C|$*, whatever* P *is.*

Proof. X is feasible, so by the converse direction of Theorem 8 (with $Z = X$) the incumbent X_P satisfies (C1)–(C4); it is therefore in the feasible region of the local maximisation, which gives $\mathrm{cov}(Y^\star) \geq \mathrm{cov}(X_P)$. Apply the objective identity. □

Remark 3 (How the code tests (C3) and (C4))*.* The implementation never materialises $\mathrm{Drop}(F)$ or $\mathrm{Src}(F)$. It observes that the global arrays `removed_` and `pin_count_` describe X, and that subtracting the cluster's own contribution turns them into a description of F:

$$\texttt{kept\_after\_revoke}(r) \equiv \neg\texttt{removed\_}[r] \;\vee\; r \in \textstyle\bigcup_{k \in X_P} T_k \quad\Longleftrightarrow\quad r \notin \mathrm{Drop}(F),$$
$$\texttt{external\_pins}(r) \equiv \texttt{pin\_count\_}[r] - \big|\{k \in X_P : r \in S_k\}\big| \quad\Longleftrightarrow\quad \big|\{f \in F : r \in S_f\}\big|.$$

Then $\texttt{avail}(y)$ requires `kept_after_revoke` on every rank of S_y — which is (C3) — and, on every rank of T_y, both `kept_after_revoke` and $\texttt{external\_pins} = 0$; the latter is (C4), and the former is redundant by Lemma 7. Computing this *without* revoking anything is what allows the solve to be non-mutating: the global state is touched only when an improvement is actually found (§5.3).

5 The algorithm

`tree-dp3` is `tree-dp` — preference forest, KEEP/COMPRESS DP, deepest-first materialization, leftover greedy, Phase II unpin/retarget — followed by the pass described here (`run_phase2_local_` in `compress.hpp`). We describe its five components in turn.

5.1 Preprocessing: caches, the dependency graph, and static caps

Candidate cache. For every $c \in \mathcal{N}$ the static list $\hat{\mathcal{K}}_c$ = `enumerate_candidates_`(c, avail = false) is computed once and sorted by coverage descending, then a descending. When the pass is invoked from `tree-dp3` this cache is handed over from the preference build, so no re-enumeration is paid. Availability is deliberately *not* used as a filter here: the cache must be valid across all the mutations the pass performs.

Dependency graph. D is built as in Definition 3: for each c and each $k \in \hat{\mathcal{K}}_c$, walk S_k and add $\{c, \mathrm{name}(r)\}$. Adjacency lists are truncated to the degree cap $d_{\max} = 16$ (`GCSA_LNS_DEGREE`). This is a heuristic restriction of the neighbourhood, not of correctness: by Corollary 9, any P whatsoever yields a sound solve.

Static caps. For every name, $\mathrm{scap}(c) := \max_{k \in \hat{\mathcal{K}}_c} \mathrm{cov}(k)$ (the head of the sorted cache, or 0). Since a feasible Y takes at most one link per name (C1), $\sum_{c \in P} \mathrm{scap}(c)$ upper-bounds $\mathrm{cov}(Y)$ for every completion Y of every frozen part. This bound is *static*: it does not depend on X, so it can be compared against the incumbent before any work is done on the cluster. This is the single most important performance device in the pass (§7).

5.2 Cluster construction

Seeds are the names of $\mathcal{N}$ in decreasing interval size, matching the greedy Phase II's bias towards large wins. For a seed c, the cluster P is the BFS ball around c in D, truncated at $q_{\max} = 8$ names (`GCSA_LNS_CLUSTER`); members must have an interval and a cache entry. Each name is used as a seed at most once per generation, but may belong to arbitrarily many clusters.

5.3 The exact solve

Algorithm 1 states the per-cluster solve. Note the following design points.

- **Incumbent seeding.** `best` is initialised to the incumbent X_P and `best_total` to $\mathrm{cov}(X_P)$, so the search can only ever return something at least as good (Corollary 9). Moreover the incumbent link of a member is *explicitly appended* to that member's option list if it is not already present. This is not redundant: a link produced by the retarget loop can carry a composite add $a > a_{\max}$ and therefore be absent from the static cache altogether, and even a cached link can be cut off by the per-name option cap. Without this step "never worse" would fail.

- **Options.** Member i gets the first $p_{\max} = 12$ (`GCSA_LNS_OPTS`) entries of $\hat{\mathcal{K}}_{c_i}$ with $\mathrm{cov} \geq \kappa$ that pass `avail` (Remark 3), i.e. conditions (C3)–(C4). Since the cache is coverage-sorted, this keeps the most valuable options.

- **Branch-and-bound.** Members are ordered by their best option coverage, descending, so the search commits to the high-value decisions first and the bound bites early. With $\mathrm{cap}(i) = \max_{y \in \mathrm{opts}(i)} \mathrm{cov}(y)$ and the suffix sums $\mathrm{suf}[k] = \sum_{j \geq k} \mathrm{cap}(\mathrm{ord}[j])$, the node at depth k with accumulated coverage tot is pruned whenever $tot + \mathrm{suf}[k] \leq best$; this is valid because at most one link per remaining member may be taken, each worth at most its cap.

Algorithm 1 SolveCluster(P) — exact re-optimization of one cluster

Input: cluster $P = (c_1, \ldots, c_q)$, current assignment X, node budget B
Output: coverage gain ≥ 0, or -1 if the budget was exhausted (state untouched)

$cur_i \leftarrow$ link of c_i in X (or none); $\quad tot_0 \leftarrow \sum_i \mathrm{cov}(cur_i)$
build Avail from X minus the cluster's own links ▷ (C3)–(C4); Remark 3
for $i \leftarrow 1$ **to** q **do**
  $\mathrm{opts}(i) \leftarrow$ first $p_{\max}$ links of $\hat{\mathcal{K}}_{c_i}$ with cov $\geq \kappa$ and Avail
  if cur_i exists and $cur_i \notin \mathrm{opts}(i)$ **then**
    append cur_i to $\mathrm{opts}(i)$ ▷ keeps the incumbent expressible
  end if
  $\mathrm{cap}(i) \leftarrow \max_{y \in \mathrm{opts}(i)} \mathrm{cov}(y)$ (0 if empty)
end for
ord $\leftarrow$ indices sorted by $\mathrm{cap}(\cdot)$ descending; $\quad \mathrm{suf}[k] \leftarrow \sum_{j \geq k} \mathrm{cap}(\mathrm{ord}[j])$
$best \leftarrow tot_0$; $\quad Y^\star \leftarrow (cur_i)_i$; $\quad nodes \leftarrow 0$
procedure Dfs(k, tot, L) ▷ L = links picked so far
  $nodes \leftarrow nodes + 1$; **if** $nodes > B$ **then abort**
  if $tot + \mathrm{suf}[k] \leq best$ **then return** ▷ suffix-sum bound
  if $k = q$ **then**
    $best \leftarrow tot$; $\quad Y^\star \leftarrow L$; $\quad$ **return**
  end if
  $i \leftarrow \mathrm{ord}[k]$
  for all $y \in \mathrm{opts}(i)$ **do**
    if y conflicts with no element of L **then** ▷ pairwise (F2)
      Dfs($k+1$, $tot + \mathrm{cov}(y)$, $L \cup \{y\}$)
    end if
  end for
  Dfs($k+1$, tot, L) ▷ leave c_i uncompressed
end procedure
Dfs(0, 0, $\emptyset$)
if aborted **then return** -1
if $best = tot_0$ **then return** 0 ▷ incumbent stands; nothing mutated
for $i \leftarrow 1$ **to** q: Revoke(c_i); $\quad$ **for** $y \in Y^\star$: Accept(y)
return $best - tot_0$

- **The "no link" branch.** Every member also has the option of taking nothing. It is tried *last*, after all concrete options, because it never improves the running total and the bound is most effective when a good incumbent has already been established. Its presence is what makes the search a search over the full downward-closed family $\mathcal{F}$ (Lemma 3) rather than over perfect matchings.

- **Internal conflicts.** The pairwise (F2) test against the currently live picks uses interval bounds as an $O(1)$ reject ($T_{y'} \subseteq [\ell_{c_{y'}}, r_{c_{y'}})$, so the covered ranks are scanned only when the source range actually overlaps that interval).

- **Non-mutating.** Nothing in the global state is touched until a strict improvement is found. On large inputs the overwhelming majority of solves end at "incumbent stands", and `revoke_`/`accept_` round trips — which rewrite `removed_`, `pin_count_` and `pin_owner_` — are the dominant cost if paid speculatively.

Algorithm 2 ClusterLNS — the `tree-dp3` Phase II pass

Input: intervals, current assignment X (from the forest DP + retarget loop)

build $\hat{\mathcal{K}}_c$ for all $c \in \mathcal{N}$ (reused from the preference build if available)
build the dependency graph D (Def. 3), truncating degrees to $d_{\max}$
$\mathrm{scap}(c) \leftarrow \max_{k \in \hat{\mathcal{K}}_c} \mathrm{cov}(k)$ for all c
dirty $\leftarrow \mathcal{N}$ sorted by isize descending
for *gen* $\leftarrow 1$ **to** G **while** *dirty* $\neq \emptyset$ **do**
    next $\leftarrow \emptyset$
    for all seeds $c \in$ *dirty*, each seed used once **do**
        $P \leftarrow$ BFS ball around c in D, capped at $q_{\max}$ names
        if $\sum_{d \in P} \mathrm{scap}(d) \leq \mathrm{cov}(X_P)$ **then**
            continue ▷ static precheck: no completion can improve
        end if
        $g \leftarrow$ SolveCluster(P)
        if $g < 0$ **and** $|P| > q_{\mathrm{fb}}$ **then**
            $P \leftarrow$ first q_{fb} members of P; $g \leftarrow$ SolveCluster(P)
        end if
        if $g > 0$ **then**
            next $\leftarrow$ *next* $\cup P \cup N_D(P)$
        end if
    end for
    dirty $\leftarrow$ *next*
end for

5.4 The driver

Algorithm 2 wraps the solve in the precheck, the fallback, and the dirty-set generation loop.

The $\sum$scap $\leq$ incumbent precheck. Before a cluster is touched, the pass compares $\sum_{c \in P} \mathrm{scap}(c)$ against $\mathrm{cov}(X_P)$. If the static upper bound does not strictly exceed the incumbent, no completion can improve, and the cluster is skipped without building Avail, without filtering options, and without entering the search. This is exact — it never skips an improvable cluster — and, empirically, it removes the great majority of the work (§7).

The size-4 retry. A cluster whose search exhausts the node budget $B = 20000$ (`GCSA_LNS_NODES`) is not abandoned. It is truncated to its first $q_{\mathrm{fb}} = 4$ BFS members (`kLnsFallbackCluster`) and re-solved. Skipping outright would forfeit the improvements a sub-cluster would still have found, and by Corollary 9 the smaller solve is just as sound. Only if the retry also aborts is the cluster left alone.

Generations. When a cluster improves, its members and their D-neighbours are marked dirty for the next generation, since their option sets and their frozen context have changed. Generations run until the dirty set is empty (a fixed point) or the shared Phase II budget (default 100, `kPhase2DefaultIters`) is spent. In practice one to three generations suffice.

5.5 Correctness

Theorem 10 (Soundness and monotonicity)**.** *Every intermediate assignment produced by Algorithm 2 is feasible, and $|C|$ is non-increasing over the run. Moreover, each applied solve returns an assignment that is* optimal *among all completions of its frozen part that are expressible in the member option lists.*

Proof. The pass starts from a feasible X (the output of the forest DP, leftover greedy and retarget loop, each of which maintains feasibility). A solve applies $F \cup Y^\star$ where $Y^\star$ satisfies (C1) by construction (one option per member, options are valid links of member names), (C2) by the explicit pairwise conflict test, and (C3)–(C4) by the AVAIL filter (Remark 3). By Theorem 8, $F \cup Y^\star$ is feasible; by Corollary 9 and the incumbent seeding of line 11 of Algorithm 1, $\mathrm{cov}(Y^\star) \geq \mathrm{cov}(X_P)$, so $|C|$ does not increase. Optimality within the option lists is the exhaustiveness of the DFS: it enumerates every element of $\prod_i(\mathrm{opts}(i) \cup \{\perp\})$ except those cut by the bound $tot + \mathrm{suf}[k] \leq best$, which discards only assignments whose total cannot strictly exceed the incumbent best, and those cut by a pairwise conflict, which are infeasible. If the node budget is exhausted the solve returns -1 and nothing is mutated. □

Note what Theorem 10 does *not* claim: global optimality. The pass is a large-neighbourhood search — exact within a neighbourhood, heuristic in the choice of neighbourhoods — and its fixed point is a local optimum with respect to the family of BFS balls of size $\leq q_{\max}$. Section 8 measures how far that falls short of the ILP optimum in practice (usually: not at all, on instances small enough to check).

6 Why it composes with, rather than replaces, the retarget loop

Because a cluster solve can never increase $|C|$, it is safe to run *after* anything. The interesting question is whether it should *replace* the pre-existing Phase II unpin/retarget loop, which is the more expensive of the two. It should not.

The retarget loop does something the cluster solve structurally cannot. When it accepts a new link w for some name and a dependent D was sourcing from ranks that w now covers, it *rewrites* D to point at the corresponding sub-run of S_w with

$$a'_D \;=\; a_D + a_w$$

(`try_accept_with_retarget_`: `d.add += cand.add`). The synthesized link is a legitimate element of $\mathcal{K}$ — (V1) holds because the two shifts compose — but it is *not* an element of any $\hat{\mathcal{K}}_c$: `enumerate_candidates_` only ever emits $a \leq a_{\max}$, whereas a'_D may exceed $a_{\max}$, and even when it does not, the composite source run need not be a maximal lcp $\geq a+1$ run. The cluster solve chooses from the static cache; it can preserve such a link (that is exactly what the incumbent appending of Algorithm 1, line 6, is for) but it can never invent one.

On long repeats this matters a great deal, because composite adds are how a deep chain of repeated motifs gets collapsed onto a single stored prefix. Running the cluster solve *instead of* the retarget loop (`GCSA_LNS_ONLY=1`) loses ground on exactly those inputs: over the 139-configuration suite at $a_{\max} = 8$,

Phase II configuration	total $\lvert C\rvert$	vs. `tree-dp`
retarget loop only (`tree-dp`)	252397	—
cluster solve only (`GCSA_LNS_ONLY=1`)	253675	+1278
retarget loop then cluster solve (`tree-dp3`)	252394	−3

The two passes are therefore complementary, not competing: the retarget loop enlarges the link universe along composite-add chains, and the cluster solve makes exact choices within the static universe. The composition is safe in one direction only — the solve after the loop, never in place of it.

7 Complexity

Preprocessing. Building D costs one pass over the source runs of all cached candidates, $O\big(\sum_c \sum_{k\in\hat{\mathcal{K}}_c} |S_k|\big)$, plus the linear-scan deduplication of adjacency lists, which is $O(d_{\max})$ per insertion after the degree cap. Static caps are $O(|\mathcal{N}|)$, since the caches are already coverage-sorted.

One cluster. A cluster has $q \le q_{\max}$ members with at most $p_{\max}$ options each, plus the "no link" branch, so the raw search tree has at most

$$(p_{\max}+1)^{q_{\max}} \;=\; 13^8 \;\approx\; 8.2\times 10^8$$

leaves in the worst case — which is why the node budget B exists at all. Each visited node performs $O(q)$ conflict tests, each $O(1)$ except on a genuine source/interval overlap, where it is $O(\mathrm{cov})$. In practice the suffix-sum bound and the descending-capacity member order collapse this by orders of magnitude: the very first root-to-leaf path takes the highest-capacity option of every member, which typically establishes a bound close to suf[0] and prunes almost everything else. The budget therefore caps the pathological cases rather than the typical ones; the fallback then re-solves at $q = 4$, i.e. at most $13^4 \approx 2.9\times 10^4$ leaves.

The pass. Each generation seeds at most $|\mathcal{N}|$ clusters; cluster construction is $O(q_{\max}d_{\max})$ and the precheck is $O(q_{\max})$. So the per-generation cost is

$$O\big(|\mathcal{N}|\cdot q_{\max}d_{\max}\big) \;+\; \sum_{\text{clusters not skipped}} O(B\cdot q_{\max}),$$

and the number of generations is bounded by the Phase II budget G. Because a generation's dirty set is confined to the neighbourhoods of the clusters that actually improved, later generations are far cheaper than the first.

The precheck in practice. The second term above is what the $\sum\mathrm{scap} \le$ incumbent test attacks, and it attacks it very effectively. On `r300-3000.fasta` ($n = 9\,000\,000$) with shape `####.####` and $a_{\max} = 8$ there are 24034 names with $|I_c| > 1$, hence 24034 seeded clusters; the precheck skips

$$22168 \text{ of } 24034 \quad (92.2\,\%),$$

leaving 1866 actual solves, of which none improved (with $a_{\max} = 8$ the retarget loop has already saturated this instance) and none aborted. The whole pass costs 0.61 s (0.20 s of which is building D) on top of a 23.4 s retarget loop:

algorithm	Phase II	total compress	$\lvert C\rvert$
`tree-dp`	23.6 s	24.0 s	1919406
`tree-dp3`	24.0 s	24.5 s	1919406

so the exact pass is at parity with `tree-dp` on a nine-megabase input — it adds about 2 % to the compression time. Without the precheck, and with a mutating solve that pays `revoke_`/`accept_` on every cluster rather than only on improvement, the same pass was reported to take about 352 s; we were not able to reproduce that figure with the current code (the two optimizations are not switchable), and record it here as the historical motivation for both devices rather than as a measurement.

8 Experimental results

8.1 The 139-configuration suite

`compare_algos` runs five hand instances, four short repetitive instances, and a weight-30 suite of 13 shapes × 10 repetition counts of a random 200 bp motif (139 configurations in total), verifying every result against brute force and against a serialized round-trip. Totals of $|C|$ over all 139 configurations:

$a_{\max}$	greedy	dep-order	`tree-dp`	`tree-dp2`	`tree-dp3`
8	259518	252357	252397	256197	**252394**
16	255785	252175	252329	255370	**213078**
32	253709	252155	252177	252917	**225814**

Per-configuration, `tree-dp3` is never worse than `tree-dp` at any $a_{\max}$ — as Corollary 9 requires, since the two share everything up to the final pass:

$a_{\max}$	better than `tree-dp`	equal	worse	reduction in total $\lvert C\rvert$
8	2	137	0	0.001 %
16	122	17	0	15.56 %
32	122	17	0	10.46 %

8.2 The counterexamples and the ILP optimum

On small instances the exact optimum is available from `ilp_baseline`, which enumerates the full link universe of Definition 1 (all sub-runs, no maximality or lcp filter) and solves the set-packing ILP with `cbc`. The two counterexamples of §2.2–2.3, and the hardest case of the fixed suite:

instance	shape	opt	greedy	dep-order	`tree-dp`	`tree-dp2`	`tree-dp3`
$(\texttt{AC})^6$	`#.#`	9	10	9	10	10	**9**
$(\texttt{ACGT})^5$	`#.#`	11	14	11	13	13	**11**
$(\texttt{GCCTTTAAAG})^4$	`#..#`	22	25	27	27	27	**22**
note text ($n = 40$)	`##`	19	20	20	20	20	20

Over the whole ten-case fixed suite (`test_ilp_cases.sh`), `tree-dp3` attains the ILP optimum on 9 of 10 instances; the sole exception is the last row above, where every heuristic stores one position too many. The $(\texttt{GCCTTTAAAG})^4$ / `#..#` case is the most striking: the optimum needs six simultaneous links, and the cluster solve finds all six, cutting $|C|$ from `tree-dp`'s 27 to 22 — an 18.5 % reduction on that instance, and also better than greedy's 25, which no other heuristic achieves.

8.3 On the ILP being a lower bound

`ilp_baseline` reports `bounds all algos`, i.e. whether the computed optimum is ≤ every heuristic's $|C|$. This is checked, not proven. The universe it enumerates is $\{k \in \mathcal{K} : a_k \le a_{\max}\}$, and by §6 the retarget loop can synthesize links with $a > a_{\max}$; such a link is decodable and would be feasible in a larger universe, so in principle a heuristic could reach a solution the ILP never considered. We are not aware of an instance where this happens, and the randomized stress harness (`test_ilp_stress.sh`, 410 instances of length 10–39 over two alphabets and six shapes) reports no violation — but the claim is empirical. Closing it properly would require enumerating links with a up to the largest composite add any heuristic can build, which is bounded only by the text length.

9 Discussion

9.1 At the default $a_{\max}$, the exact solve is starved

The most important number in §8 is the smallest one. At $a_{\max} = 8$, exact re-optimization over bounded dependency neighbourhoods — a genuinely exact procedure, applied to every name in the instance — buys 3 stored positions across 139 configurations, all of them on the two hand-built counterexamples ($10 \to 9$ and $13 \to 11$). On all 137 real configurations the answer is unchanged.

This is not a failure of the neighbourhood size. At $a_{\max} = 8$ on the large repetitive instances, the precheck reports that for over 90 % of clusters the *static* bound $\sum_{c \in P} \mathrm{scap}(c)$ does not even exceed what the incumbent already achieves: there is no better assignment to find, no matter how large a neighbourhood one is willing to solve, because the candidate lists do not contain anything better. What binds is the *candidate universe*, not the search.

Raising $a_{\max}$ to 16 enlarges that universe and the same algorithm becomes worth 15.5 % of total $|C|$, improving 122 of 139 configurations. It is worth being precise about the division of labour here: greedy, dep-order, `tree-dp` and `tree-dp2` *all* barely move between $a_{\max} = 8$ and $a_{\max} = 16$ (totals change by under 1.5 %), because none of them can exploit the extra candidates — they commit to one preferred link per name and the preferred link rarely changes. The exact solve can, because it evaluates the whole option list against its neighbours. The lesson generalises: enriching the candidate universe pays only if the selection procedure is strong enough to use it, and strengthening the selection procedure pays only if the universe is rich enough to reward it. `tree-dp3` at $a_{\max} = 16$ is the first configuration in which both halves are in place.

9.2 The $a_{\max} = 32$ regression is a neighbourhood-size artifact

Going from $a_{\max} = 16$ to 32 makes `tree-dp3` worse (213078 → 225814) while leaving every other algorithm essentially unchanged. Since larger $a_{\max}$ strictly enlarges the candidate universe, this must be an artifact of the caps rather than of the problem. We measured which cap:

suite configuration (×100 reps, $a_{\max} = 32$)	$q_{\max} = 8$	aborts	$q_{\max} = 12$	aborts	($a_{\max} = 16$, $q_{\max} = 8$)
`5+g4+20+g4+5`	3270	0	2796	25	2717
`10+g8+20`	3270	0	2890	28	2807
`5+g2+20+g2+5`	3063	1	2679	29	2881

The node budget is not the binding constraint: at $q_{\max} = 8$ the abort count is 0 or 1, and raising `GCSA_LNS_NODES` by a factor of 20 changes nothing at all. Raising the *cluster* cap from 8 to 12 recovers most of the loss — and on one of the three configurations overshoots the $a_{\max} = 16$ result. The mechanism is straightforward once stated: a larger $a_{\max}$ gives each name candidates sourcing from more distinct intervals, so the dependency graph densifies, and a BFS ball of a fixed 8 names covers a progressively smaller fraction of each name's true dependency neighbourhood. More of the relevant structure ends up frozen at the cluster boundary, and conditions (C3)–(C4) of Theorem 8 bind harder. The regression is thus a tuning artifact of $q_{\max}$, and the natural fix is to scale $q_{\max}$ with the observed dependency degree rather than fixing it at 8.

(An earlier account attributed this regression to node-cap aborts. The measurements above do not support that: at the default cluster cap the aborts are essentially absent, and the node budget is not the active constraint.)

9.3 What remains

Three directions follow directly from the analysis above.

1. **Cluster sizing.** $q_{\max}$ is a fixed constant chosen for the $a_{\max} = 8$ regime. Given a per-instance measurement of the dependency degree it could be chosen adaptively, with the node budget — which is currently slack — absorbing the variance.

2. **Bringing composite adds into the universe.** The retarget loop's composite links are exactly the ones the static cache lacks (§6). Emitting them into $\hat{\mathcal{K}}_c$, rather than only preserving them when they happen to be incumbent, would let the exact solve reason about them — and would also close the ILP lower-bound caveat of §8.3.

3. **Beyond BFS balls.** The neighbourhood family is currently "bounded balls around each name". Since the algorithm is exact within whatever family it is given, a family chosen from the structure of the incumbent — for instance the tight cycles of the preference graph, which are precisely what §2.2 and §2.3 show `tree-dp` mishandles — might buy more per unit of work than uniform balls.

What the present note establishes is the structural point underneath all three: because the compressed array stores literal positions, the feasible region of differential compression is a plain set-packing region with pairwise constraints and no acyclicity condition, and because a link's covered ranks lie in its own interval, any set of names can be carved out and re-optimized exactly against a two-condition interface. Exactness on a neighbourhood is therefore cheap and always safe. Whether it is *useful* is entirely a question of how rich the candidate universe is, and at the default $a_{\max}$ it is not yet rich enough.